



UNIDADE II

Engenharia de Software Ágil

Prof. Me. Edson Moreno

Introdução

Esta unidade apresenta o emprego de metodologias ágeis nas atividades do processo de software, modelos de processos individuais, de equipe e corporativos.

Os modelos de processos de software representam uma visão simplificada do processo de software. Cada modelo representa uma perspectiva particular e os mais conhecidos são:

- Modelos de processos pessoal e de equipe: PSP e TSP.
- Modelos de processos tradicionais: Cascata, Balbúrdia, Prototipagem, Incremental, RAD e Espiral.
 - Os modelos de processos descrevem os serviços necessários para alinhar os recursos do projeto, do produto software e stakeholders.

Processos de software

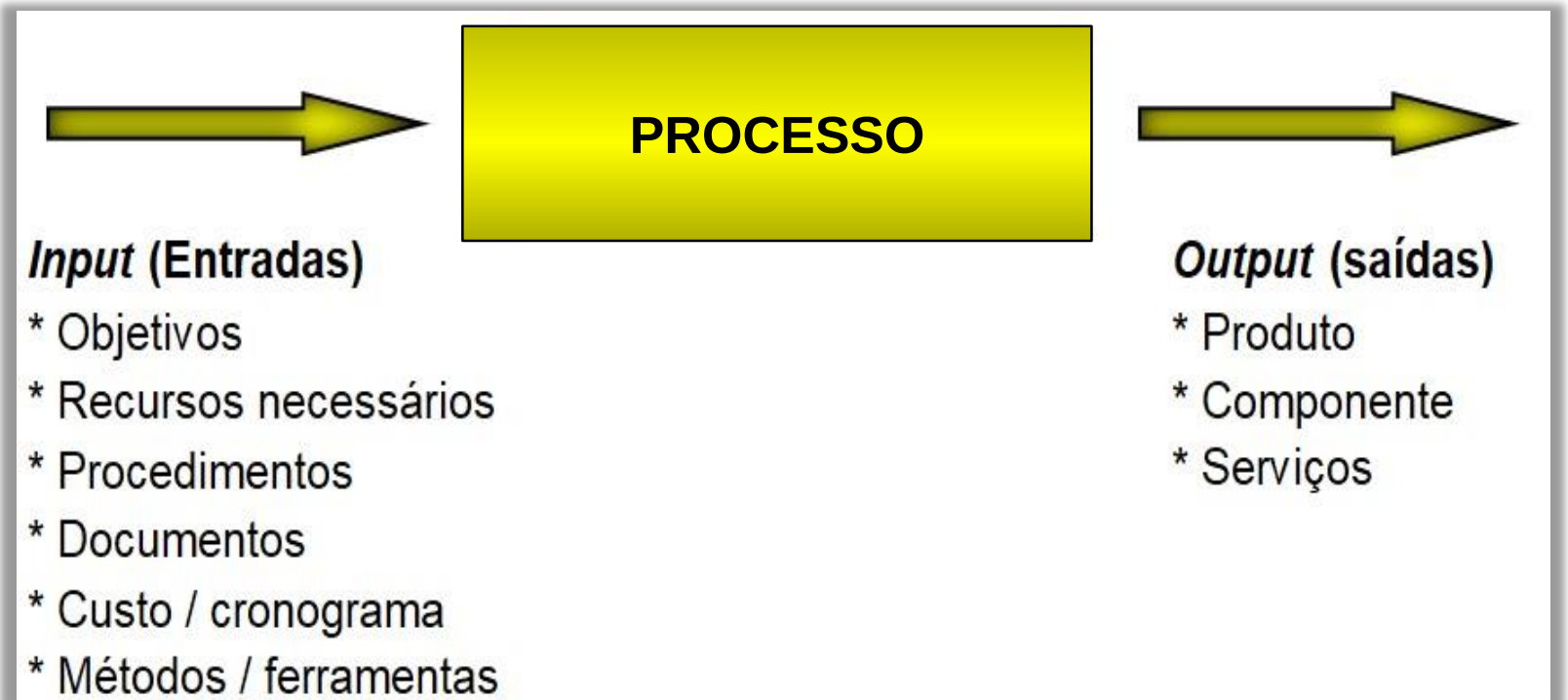
- O processo de software constitui a base para o controle do gerenciamento de projetos de software e estabelece o contexto no qual são aplicados métodos técnicos, são produzidos artefatos (modelos, documentos, dados, relatórios, formulários etc.), são estabelecidos marcos, a qualidade é garantida e as mudanças são geridas de forma apropriada (Pressman, 2025).
- Um modelo de processo de software é uma representação simplificada de um processo de software. Cada modelo representa uma perspectiva particular de um processo e, portanto, fornece informações parciais sobre ele (Sommerville, 2011).

Neste capítulo serão abordados:

- Agilidade e decomposição do processo
- PSP – Personal software process
- TSP – Team software process
- Modelos de processos de software: Cascata, Balbúrdia, Prototipagem, Incremental, RAD e Espiral.

Processos de software

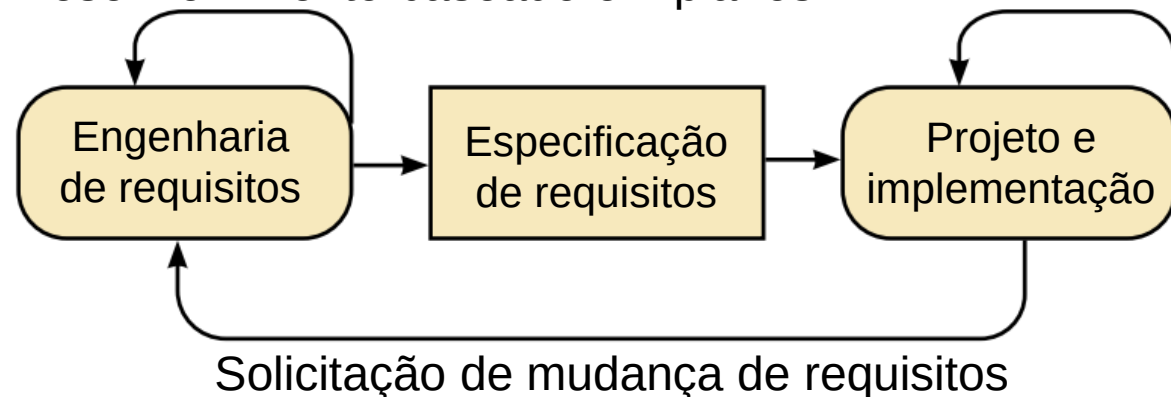
- Um processo é uma sequência de fatos, atividades ou operações que apresentam certa unidade e/ou que se reproduzem com certa regularidade (Pressman, 2002).
- O processo deve ser delimitado, definindo seu início e fim. Os pontos de início (input) e fim (output) são marcos de referências (milestones), úteis para o acompanhamento do processo.
- Os marcos de referência servem para controlar, medir, avaliar, projetar e melhorar o processo.



Agilidade e decomposição do processo

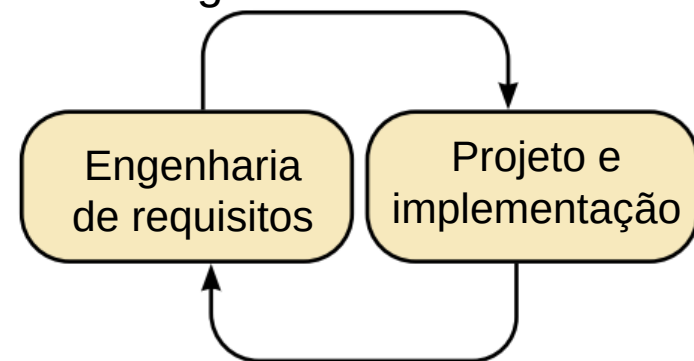
- A agilidade no desenvolvimento de software mede a capacidade de fornecer feedbacks rápidos às mudanças no software que ocorrem ao longo de seu ciclo de vida.
- Na engenharia de software, os processos são tratados de duas formas: desenvolvimento prescritivo e desenvolvimento ágil.
- O desenvolvimento prescritivo segue uma abordagem estruturada e sequencial, com etapas bem definidas, especificadas e detalhadas antes do início do projeto, geralmente utilizando como base o modelo de processo cascata.

Desenvolvimento baseado em planos



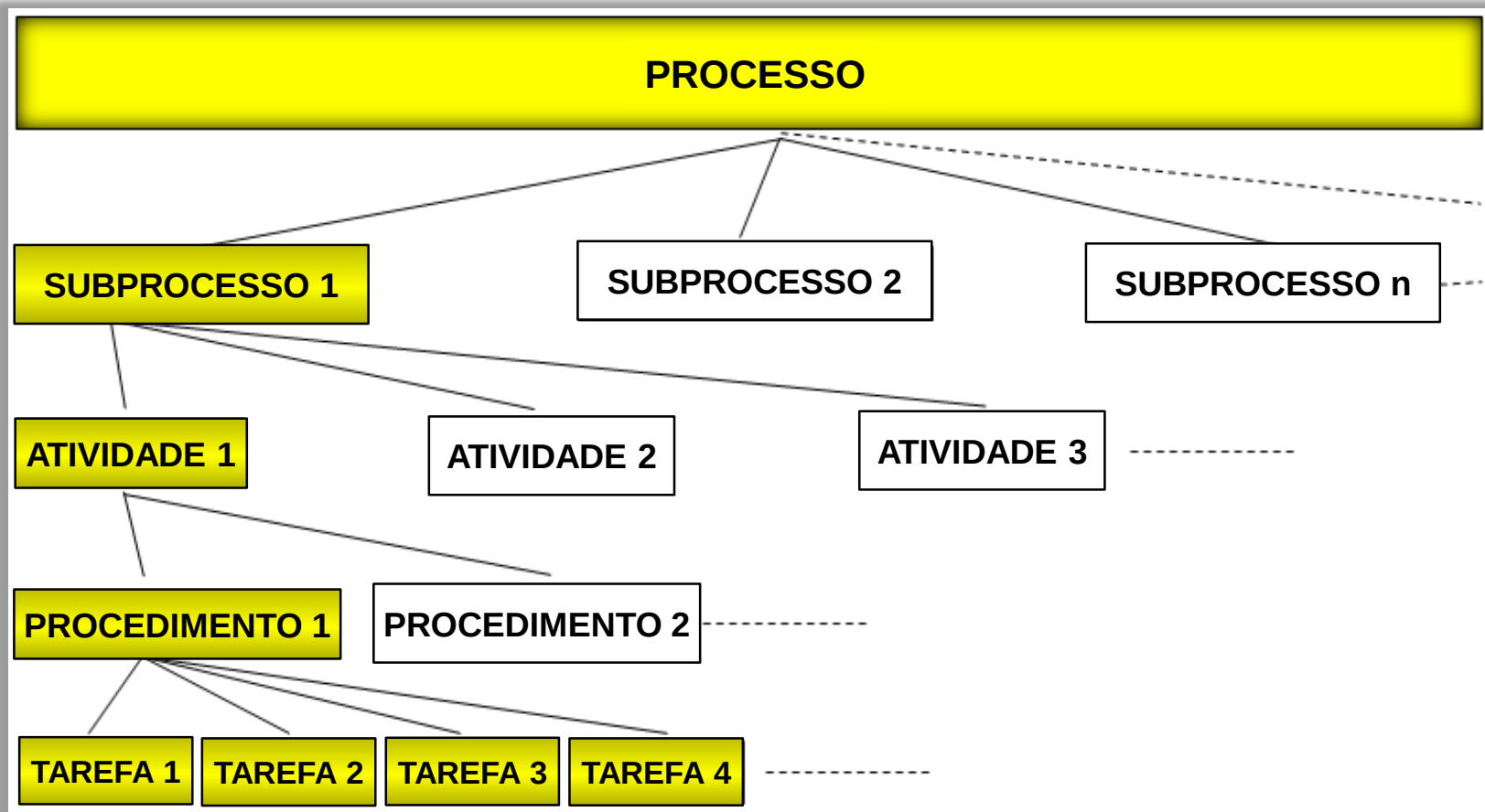
- O desenvolvimento ágil é uma abordagem iterativa e incremental, com baixo índice de especificação, que prioriza a adaptação a mudanças e a entrega de valor contínuo ao cliente.

Desenvolvimento ágil



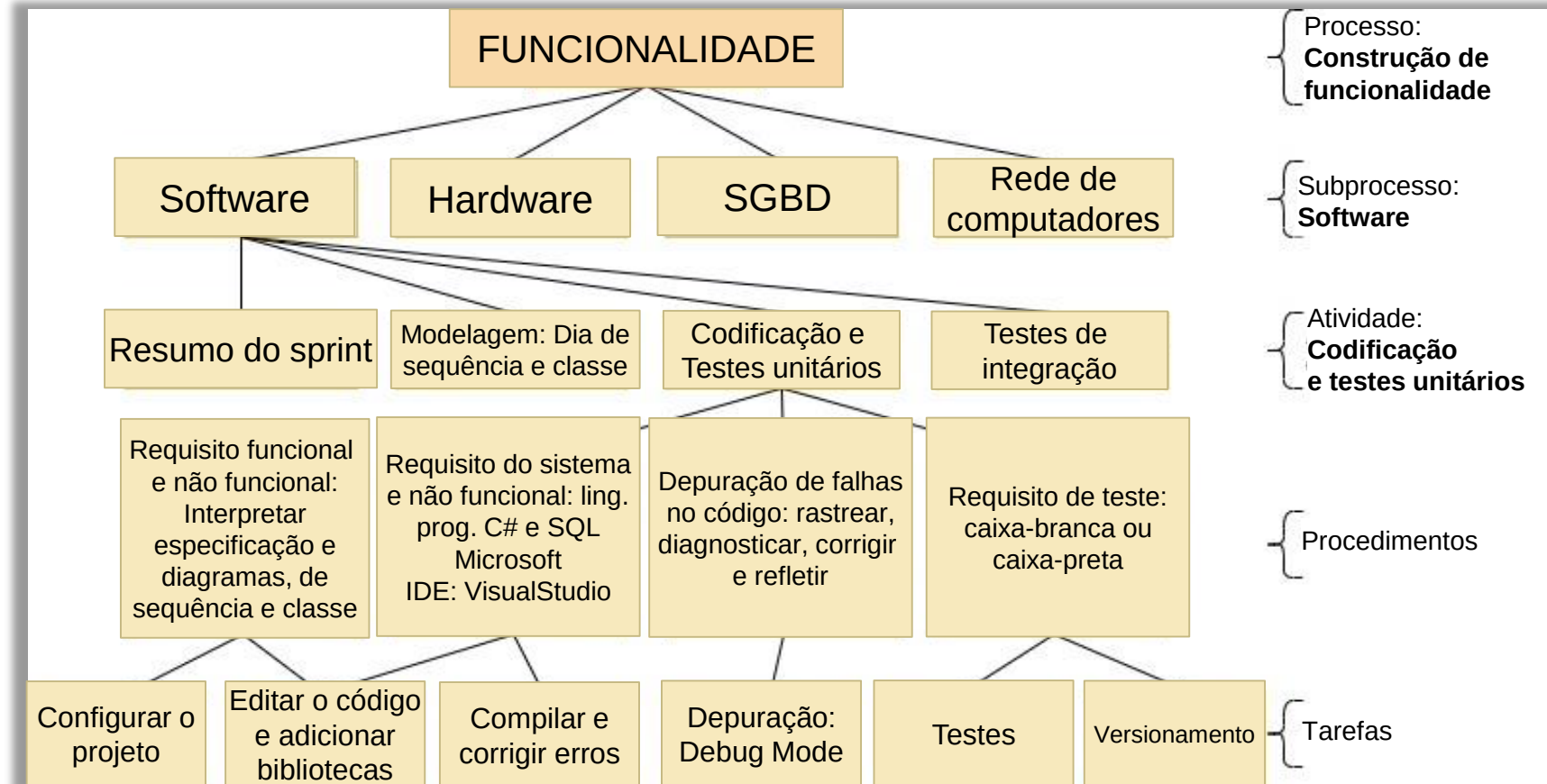
Decomposição do processo

- Decompor um processo é estabelecer uma sequência coerente de práticas, que objetiva o desenvolvimento ou evolução de sistemas de software.
- O processo de engenharia de software pode, por exemplo, representar a implementação de uma determinada funcionalidade.



Decomposição do processo de implantação de uma funcionalidade

- O modelo abaixo mostra apenas o desenvolvimento da atividade “Codificação e Testes Unitários”.
- Cada subprocesso é decomposto, servindo de guia para o processo de desenvolvimento, trazendo maior clareza e uma maior flexibilidade no acompanhamento do processo, podendo ser formado por guias de cartões (kanban).



PSP – Personal Software Process (Processo de Software Pessoal)

O PSP é um processo de desenvolvimento específico para o engenheiro de software.

- Ele orienta o planejamento e o desenvolvimento dos componentes do software.
- ➔ A ideia é: “A melhoria da capacidade da organização depende da melhoria de cada indivíduo”.

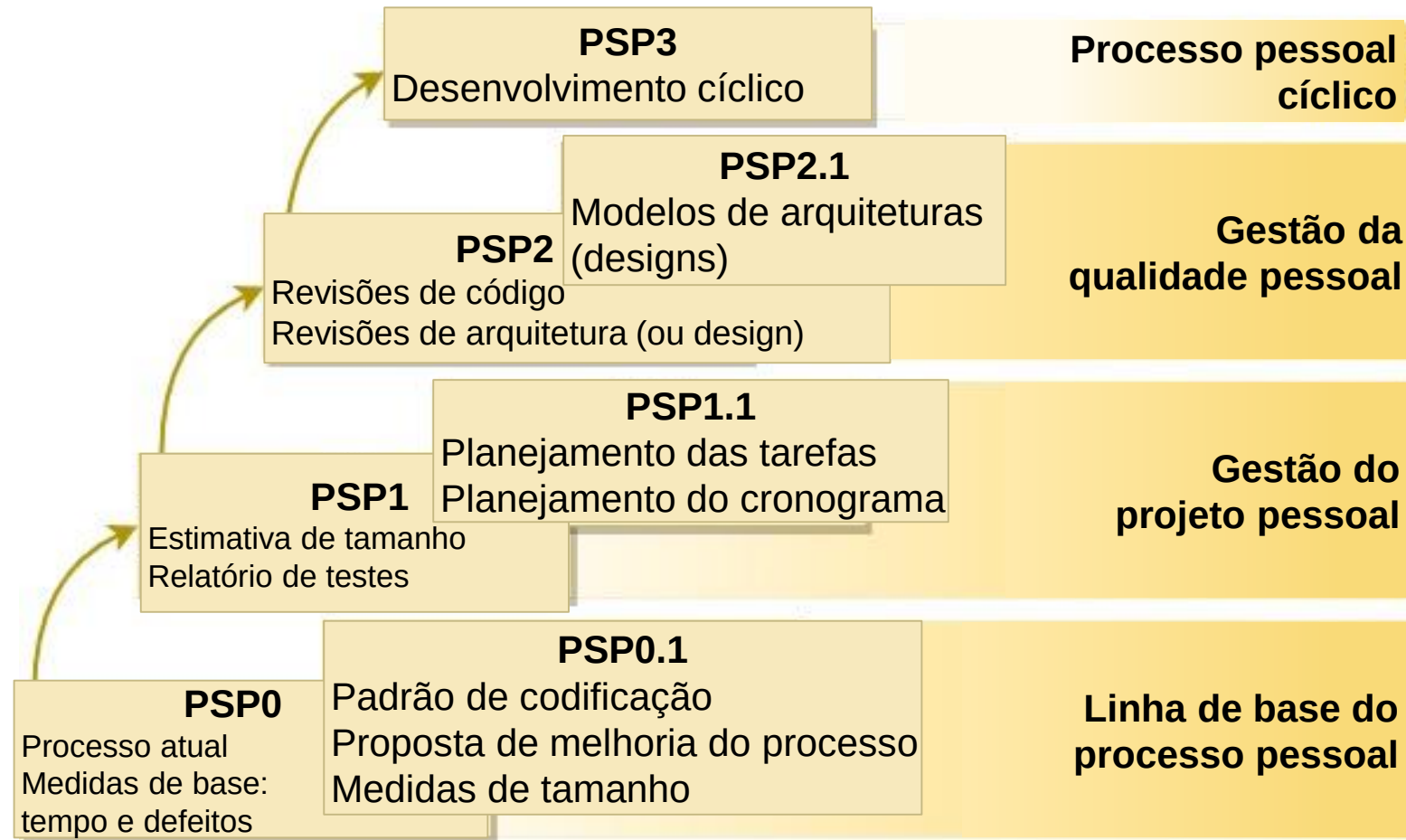


Figura – Níveis do processo PSP

Fonte: Hayes (1997), tradução: Moreno (2025).

TSP – Team Software Process (Processo de Software da Equipe)

- O TSP é uma extensão do PSP com enfoque na equipe de trabalho.
- Ele foi “criado” para ser seguido por desenvolvedores treinados no PSP, para trabalhar em equipes auto-organizadas e desenvolver softwares de qualidade.
- O TSP é uma solução para pequenas organizações de software, que se consideram pequenas para enfrentar as complexidades do CMMI.

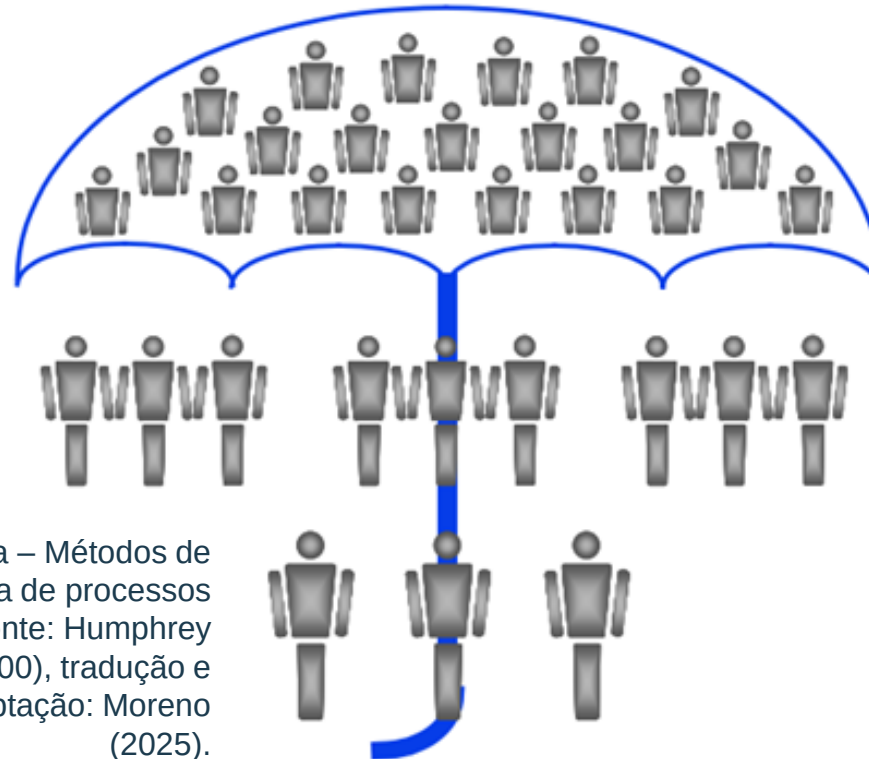


Figura – Métodos de melhoria de processos
Fonte: Humphrey (2000), tradução e adaptação: Moreno (2025).

CMMI – Melhora a capacidade da organização; foco na gestão.

TSP – Melhora o desempenho da equipe; foco na equipe e no produto.

PSP – Melhora as habilidades individuais e a disciplina; foco pessoal.

TSP – Elementos fundamentais

- O TSP fornece um framework para a equipe de software planejar e gerenciar seus próprios processos de trabalho, acompanha o progresso e gerencia as tarefas do dia a dia como Scripts de processos; formulários, relatórios e gráficos; papéis e responsabilidades.
- O TSP oferece um conjunto de elementos fundamentais para ajudar as equipes de desenvolvimento a entregar software de alta qualidade de forma previsível e eficiente.

Elementos Fundamentais do TSP	
1. Papéis e Responsabilidades	6. Comunicação e Coordenação
2. Planejamento Detalhado	7. Gerenciamento de Riscos
3. Ciclos de Desenvolvimento	8. Incidentes e Defeitos
4. Garantia da Qualidade	9. Revisão e Melhoria Contínua (iteração)
5. Métricas e Monitoramento	10. Documentação

Interatividade

O TSP (Team Software Process) é um modelo criado para auxiliar equipes de desenvolvimento de software a atingirem altos níveis de desempenho e qualidade. Esse processo propõe práticas organizadas que ajudam as equipes a planejar, monitorar e melhorar continuamente seu trabalho. Com base no TSP, assinale a alternativa correta:

- a) O principal foco do TSP é na automação de testes e ferramentas de versionamento, não sendo aconselhável para a gestão de equipes.
- b) O TSP é uma abordagem ágil que não necessita de planejamento, porque adota uma abordagem puramente adaptativa e sem documentação.
- c) O TSP é aplicado em projetos pessoais que possui relação com o modelo CMMI.
 - d) O TSP incentiva equipes a seguirem práticas disciplinadas de planejamento, estimativas, medição e melhoria contínua.
 - e) O TSP substitui os métodos ágeis como Scrum e XP.

Resposta

O TSP (Team Software Process) é um modelo criado para auxiliar equipes de desenvolvimento de software a atingirem altos níveis de desempenho e qualidade. Esse processo propõe práticas organizadas que ajudam as equipes a planejar, monitorar e melhorar continuamente seu trabalho. Com base no TSP, assinale a alternativa correta:

- a) O principal foco do TSP é na automação de testes e ferramentas de versionamento, não sendo aconselhável para a gestão de equipes.
- b) O TSP é uma abordagem ágil que não necessita de planejamento, porque adota uma abordagem puramente adaptativa e sem documentação.
- c) O TSP é aplicado em projetos pessoais que possui relação com o modelo CMMI.
- d) O TSP incentiva equipes a seguirem práticas disciplinadas de planejamento, estimativas, medição e melhoria contínua.
- e) O TSP substitui os métodos ágeis como Scrum e XP.

Modelos de processos de software

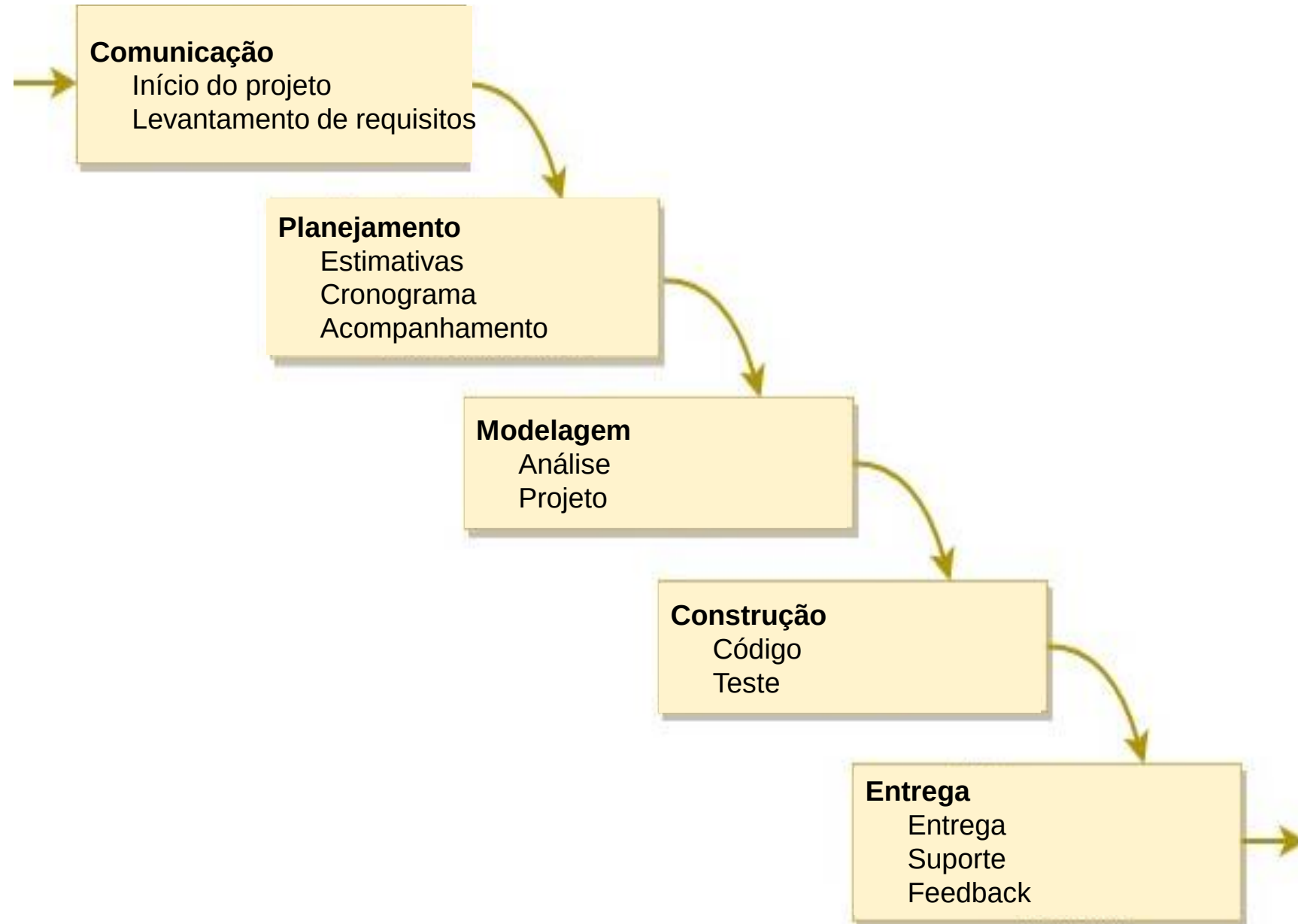
- Um modelo de processo de software engloba um conjunto de atividades, métodos, práticas e transformações empregadas no desenvolvimento e manutenção de software.
- Para o projeto ou partes específicas do projeto, a adoção de um modelo de processo de software permite ter uma visão geral do sistema como um processo orientado por uma estrutura organizacional.

Os principais modelos de processo de software, alguns conhecidos também por modelos prescritivos, são mostrados na lista abaixo:

- Modelos de processos: pessoal (PSP) e de equipe (TSP).
- Modelos de processos tradicionais: Cascata, Balbúrdia, Prototipagem, Incremental, RAD e Espiral.
- Processo Unificado: RUP, Praxis e ICONIX.

Modelo de processo de software: Cascata (Waterfall ou Sequential Linear)

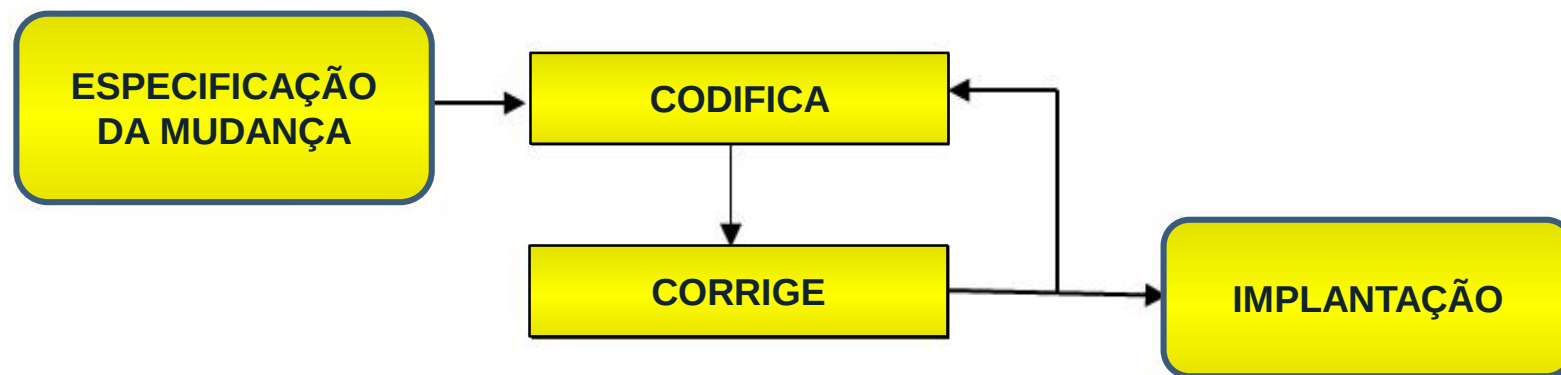
- Modelo de processo
Cascata (Waterfall ou
Sequential Linear)
Considerado o modelo clássico de vida do desenvolvimento do software e serve de base para os demais modelos.



Modelos de processo de software: Balbúrdia

- Modelo de processo Balbúrdia (Codifica/Corrige ou Codifica/Remenda) – Neste modelo não há planejamento, nem controle com relação aos possíveis riscos.
- Ele é caracterizado pela administração do caos, pela informalidade, pelo loop de gestão Codifica/Corrige, em que planejamento, requisitos do software, modelagem, testes e documentação são caóticos ou até mesmo inexistentes.

O modelo balbúrdia não é propriamente um modelo de processo de software a ser seguido. Entra aqui em destaque porque é uma prática comum de desenvolvedores, e na verdade é um modelo que deve ser evitado.



Modelo de processo de software: Prototipagem

Prototipagem – É simular comportamento e aparência do sistema final por meio de protótipos.

- O protótipo de software é um mecanismo para identificar requisitos de interface humano-computador (IHC), identificar requisitos funcionais (RF), práticas de experiência do usuário (UX) em mapas de navegação interativos, mapa de estilos para CSS e IDEs.

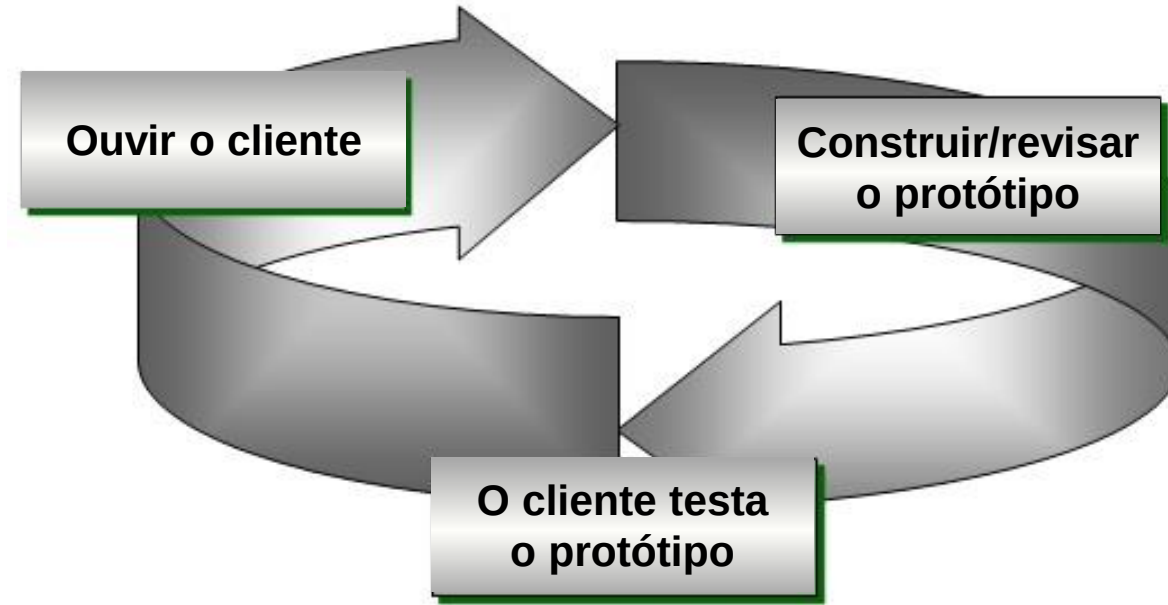


Figura – O paradigma de prototipagem
Fonte: Pressman (2011).



Figura – Design de interface do usuário
Fonte: Patel (2025).

Modelo de processo de software: Prototipagem – Design de IHC (Mockup)

- O desenvolvimento da aparência visual da tela inclui cores, tipografia, imagens e outros elementos gráficos para dar vida ao design.
- Este protótipo é chamado de Mockup (visual detalhado) para a construção da tela.
- Nota: As medidas de tela e de figuras como caixas de imagem, de textos e mídias são em pixels (px).

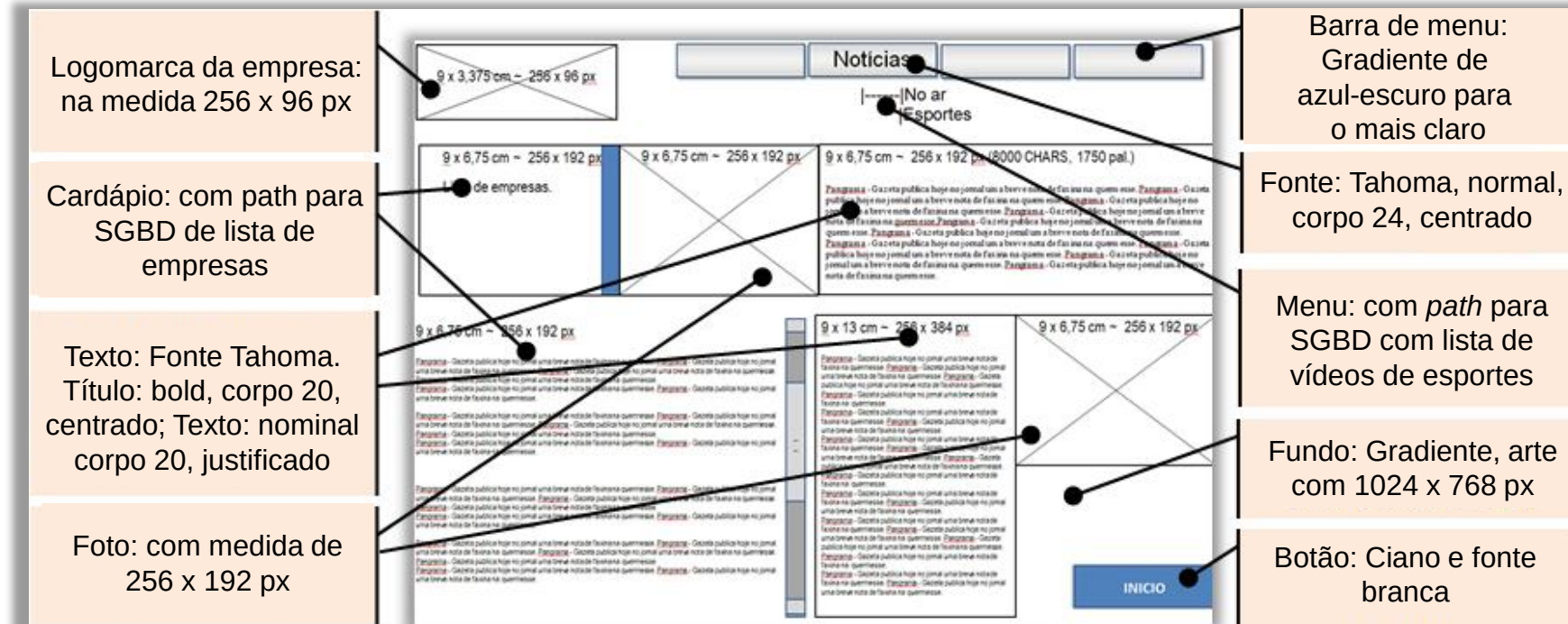
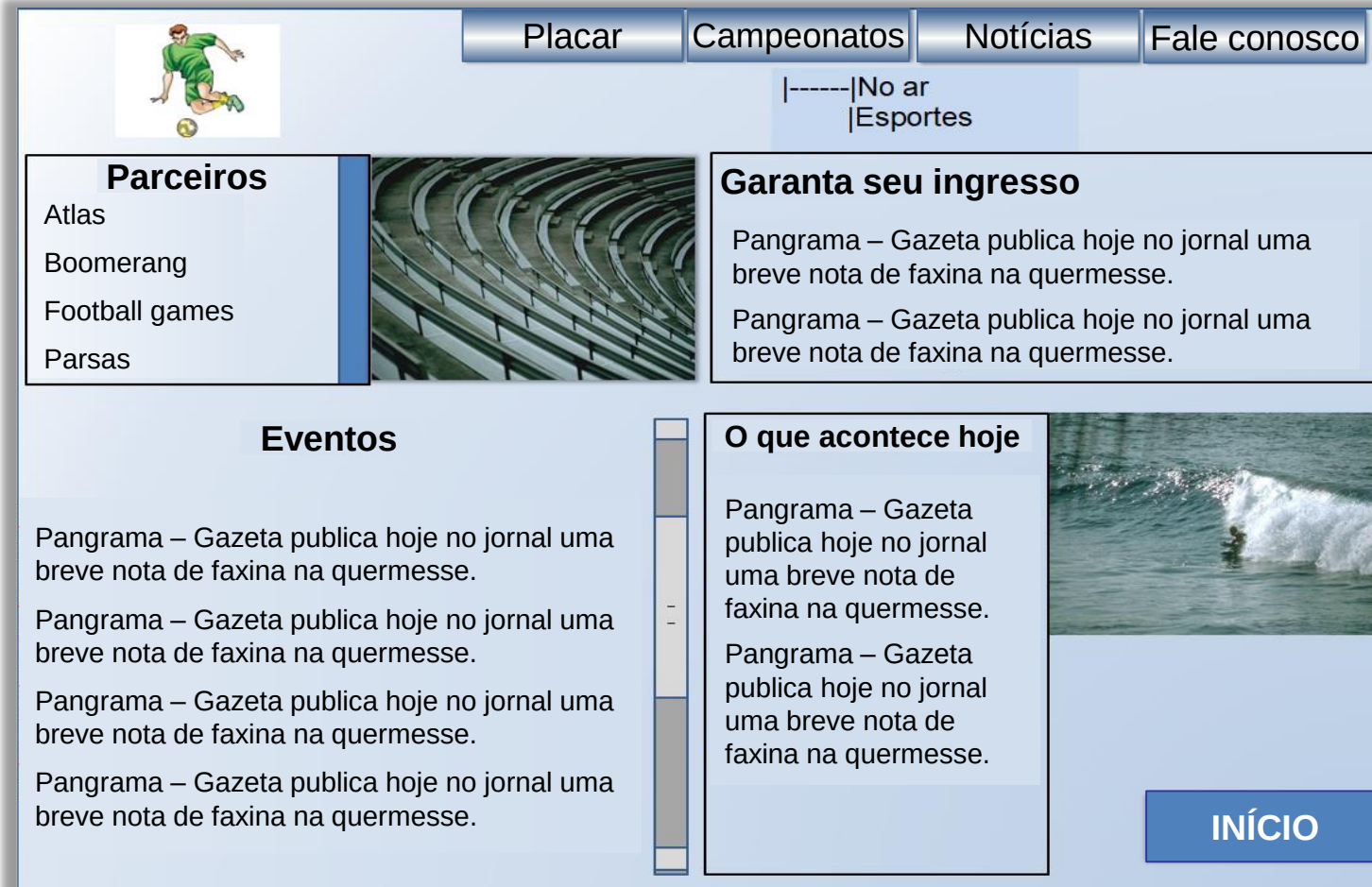


Figura – Mockup de tela de software
Fonte: Moreno (2025).

Modelo de processo de software: Prototipagem – Design de IHC (Layout)

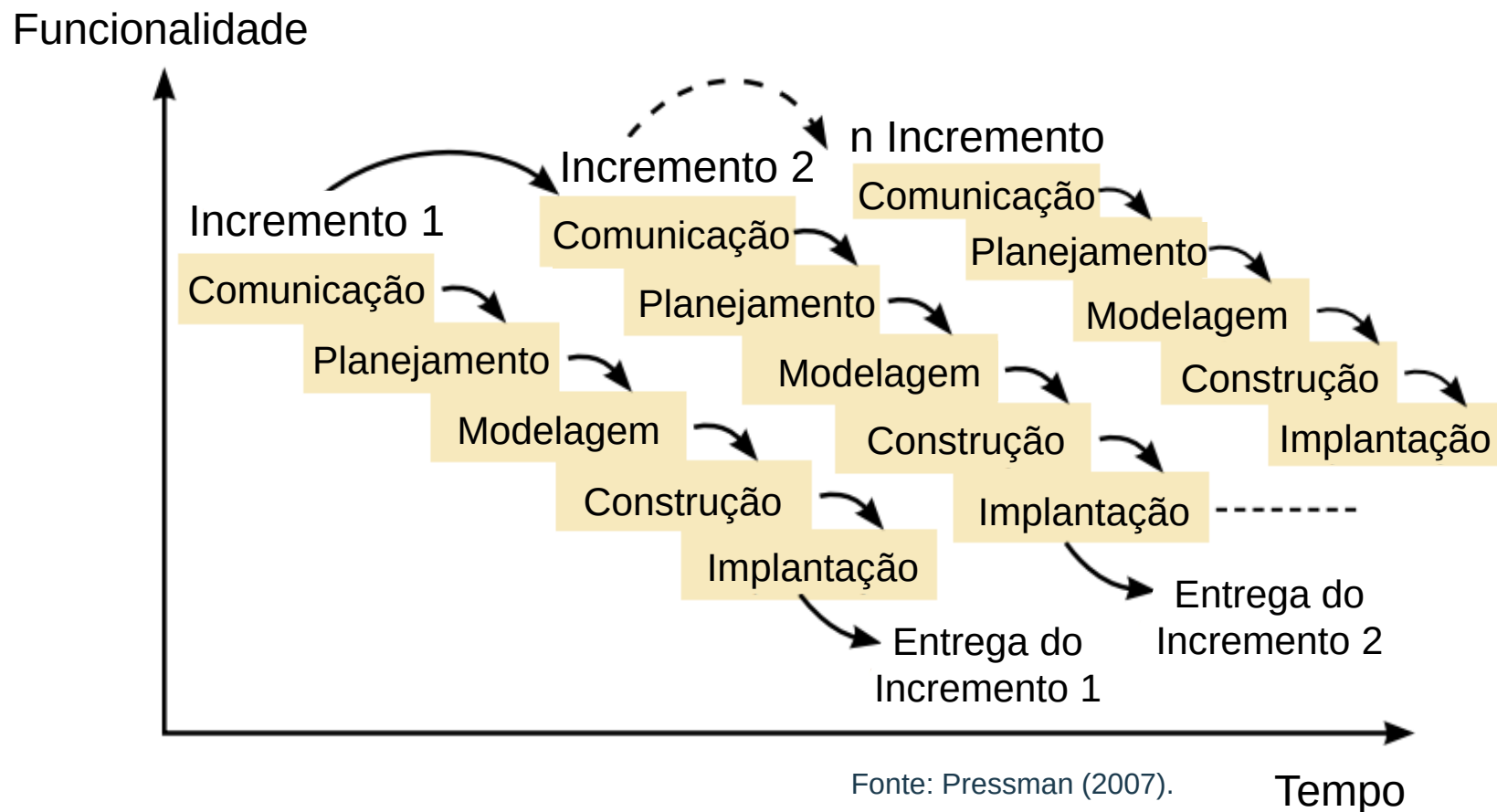
- Após o mockup, os elementos de imagem são inseridos, de acordo com os requisitos do mockup.
- É construído o layout de tela (design final).
- Com ferramentas para layout ou IDE é feita a combinação do mockup com aspectos funcionais do software, considerando a interatividade, navegabilidade e a experiência do usuário.

Figura – Layout de tela de software
Fonte: Moreno (2025).



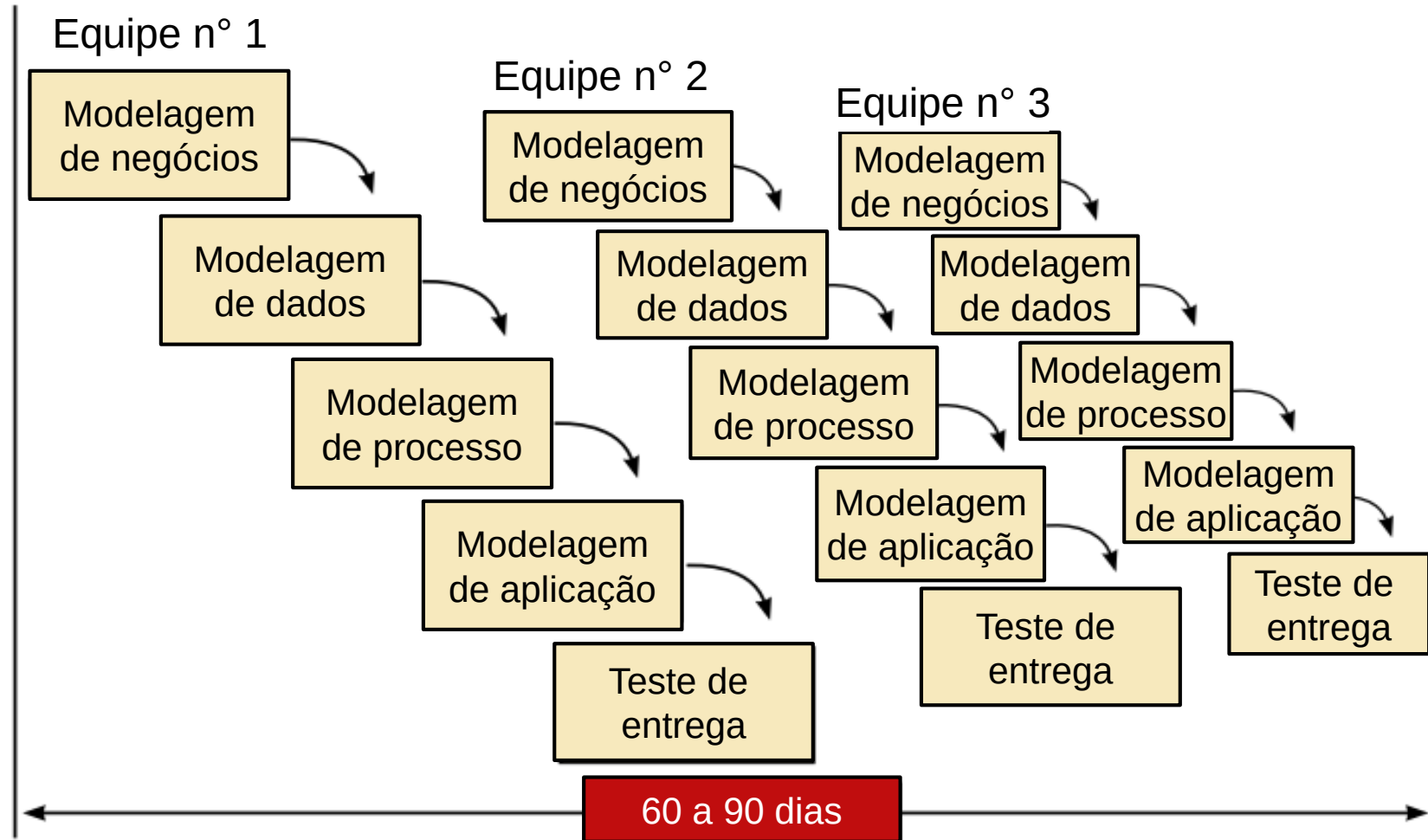
Modelo de processo de software: Incremental

- Modelo de processo Incremental – É um modelo evolucionário com sequências lineares dos elementos cascata aplicados para desenvolver uma determinada funcionalidade.
- Esse modelo é caracterizado por ser iterativo e ter como objetivo apresentar um produto operacional a cada incremento realizado, ou seja, cada incremento representa uma versão do software.



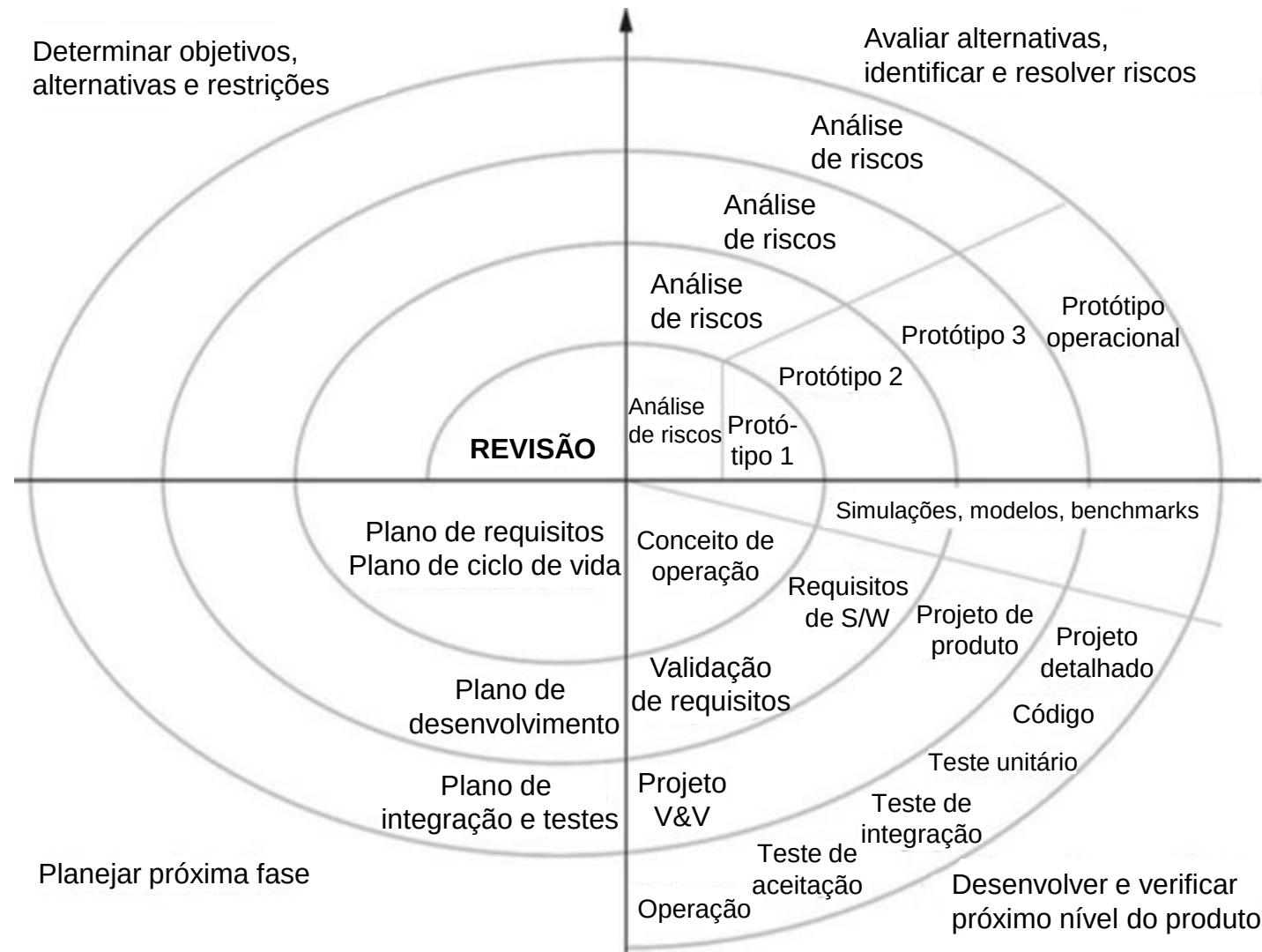
Modelo de processo de software: RAD

- Modelo de processo RAD (Rapid Application Development) – É um modelo misto com característica de vários outros modelos e tem como objetivo a entrega rápida do software.
- O RAD possui uma abordagem baseada na construção de componentes.
- Cada função é direcionada para uma equipe, para então serem integradas para formar o software.



Modelo de processo de software: Espiral

- Modelo de processo Espiral – É um modelo evolucionário que combina a natureza iterativa da prototipagem com o estilo clássico do modelo cascata.
- O software é desenvolvido em uma série de versões evolucionárias e a cada ciclo é definido um conjunto de atividades.
- Um release é definido e após várias iterações o software atinge sua totalidade.



Fonte:
Sommerville
(2016).

Interatividade

A seguir estão descrições de três modelos de processos de software:

- I. Modelo iterativo e rápido voltado à construção de aplicações com forte envolvimento do usuário e uso de componentes reutilizáveis.
- II. Modelo que combina desenvolvimento incremental com análise contínua de riscos.
- III. Modelo tradicional que segue etapas sequenciais, em que cada fase deve ser concluída antes da próxima ser iniciada.

Assinale a alternativa que corresponde à sequência correta dos modelos descritos:

- a) Cascata, Espiral e RAD.
- b) Incremental, Cascata e Balbúrdia.
- c) Incremental, Prototipação e Espiral.
- d) RAD, Espiral e Cascata.
- e) RAD, Incremental e Espiral.

Resposta

A seguir estão descrições de três modelos de processos de software:

- I. Modelo iterativo e rápido voltado à construção de aplicações com forte envolvimento do usuário e uso de componentes reutilizáveis.
- II. Modelo que combina desenvolvimento incremental com análise contínua de riscos.
- III. Modelo tradicional que segue etapas sequenciais, em que cada fase deve ser concluída antes da próxima ser iniciada.

Assinale a alternativa que corresponde à sequência correta dos modelos descritos:

- a) Cascata, Espiral e RAD.
- b) Incremental, Cascata e Balbúrdia.
- c) Incremental, Prototipação e Espiral.
- d) RAD, Espiral e Cascata.
- e) RAD, Incremental e Espiral.

Introdução

- Cada modelo de processo representa o processo em uma perspectiva particular, que depende do tamanho, complexidade e qualidade exigida para software.
- Por exemplo: O modelo incremental é acompanhado do versionamento, que acompanha a evolução ou mudanças do software por meio de registros de entrega, de uma funcionalidade por exemplo.
- No caso do modelo incremental são apresentadas as atividades sequenciais desenvolvidas ao longo de seu ciclo, no entanto não mostra a definição dos papéis e responsabilidades das pessoas envolvidas no projeto.
 - O planejamento do processo de desenvolvimento acompanha cada etapa do Ciclo de Vida de Desenvolvimento de Software (do inglês, Software Development Life Cycle (SDLC)).
 - Após a escolha do modelo apropriado, selecionam-se os artefatos a serem produzidos, alinhando as pessoas com as atividades do desenvolvimento, ditadas pelo modelo de processo escolhido.

Planejamento do processo de software

- O planejamento do processo de software refere-se à definição detalhada das atividades, metodologias e práticas a serem adotadas ao longo do ciclo de vida do desenvolvimento de software.
- O objetivo é garantir a entrega de um produto de alta qualidade, dentro dos requisitos de tempo e custo estipulados.
- Um modelo de processo de software é uma representação simplificada das atividades do processo de software, que acompanham todo o Ciclo de Vida de Desenvolvimento de Software (Sommerville, 2016).

Neste capítulo serão abordados:

- Software Development Life Cycle (SDLC)
- Aspectos humanos da engenharia de software
- Análise de recursos para os processos do projeto
- Métodos, ferramentas e técnicas (MF&T): Modelagem da infraestrutura de TI

Software Development Life Cycle (SDLC)

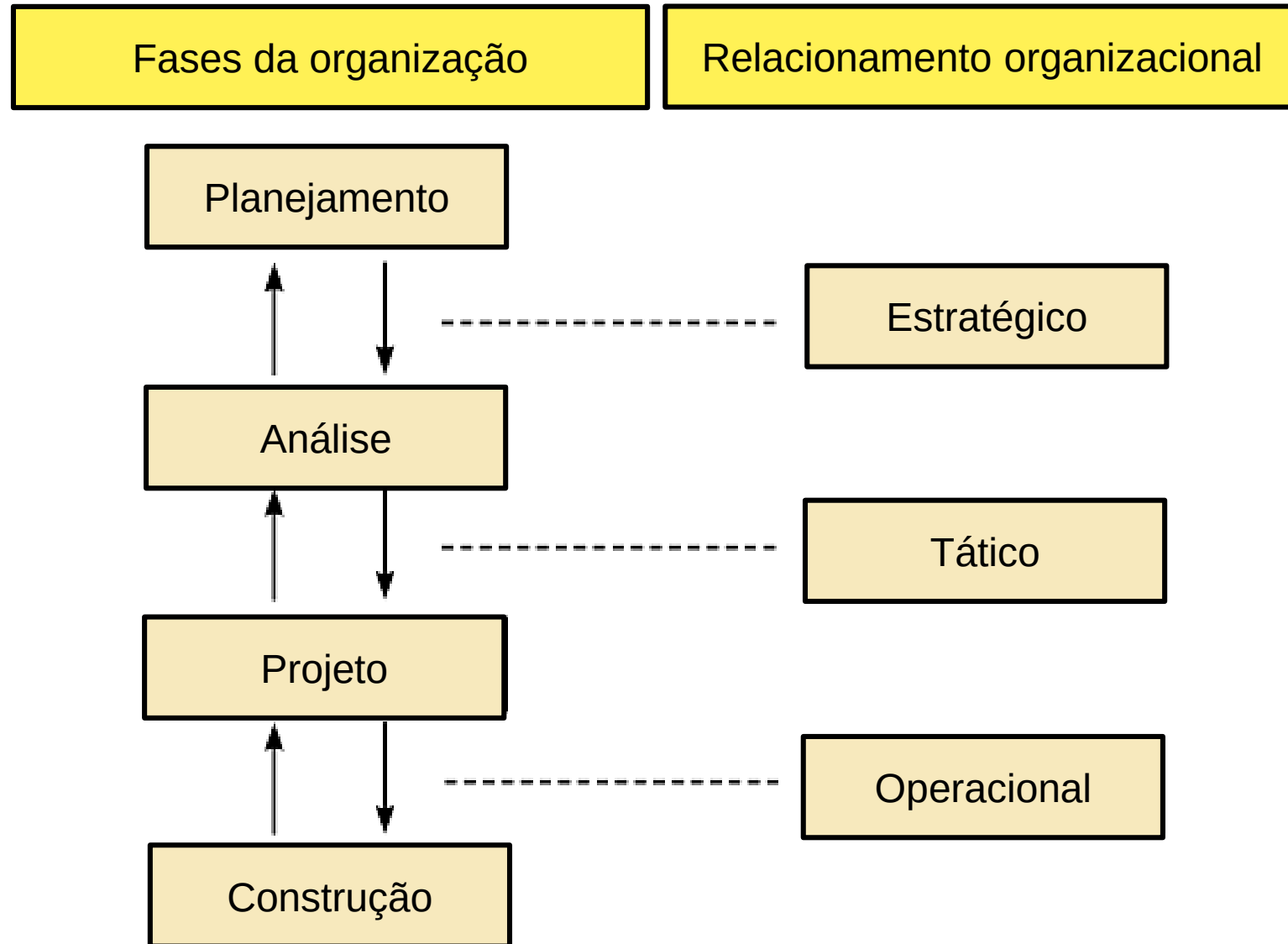
Software Development Life Cycle – SDLC (Ciclo de Vida de Desenvolvimento de Software) – Um exemplo de ciclo de vida do software é apresentado pela empresa Arkan.

- O ciclo de vida tem base na qualidade e abrange várias fases, desde a concepção, design, implementação, testes, implantação, manutenção e suporte.



Estrutura organizacional

- O alinhamento do SDLC com estruturas organizacionais envolve as atividades, funções (papéis) das pessoas, responsabilidades e fluxos de trabalho dentro da organização do processo de desenvolvimento.
- A estrutura organizacional influencia o grau de autoridade e responsabilidades.



Aspectos humanos da engenharia de software

- O software é desenvolvido por pessoas, usado por pessoas e dá suporte à interação entre pessoas. Assim, características, comportamento e cooperação humanos são fundamentais no desenvolvimento prático de software (Pressman, 2021).
- O engenheiro de software é quem precisa dominar uma diversidade de processos, métodos, ferramentas e técnicas da engenharia de software, desenvolver habilidades para entender problemas ou necessidades de software capazes de projetar soluções com qualidade.

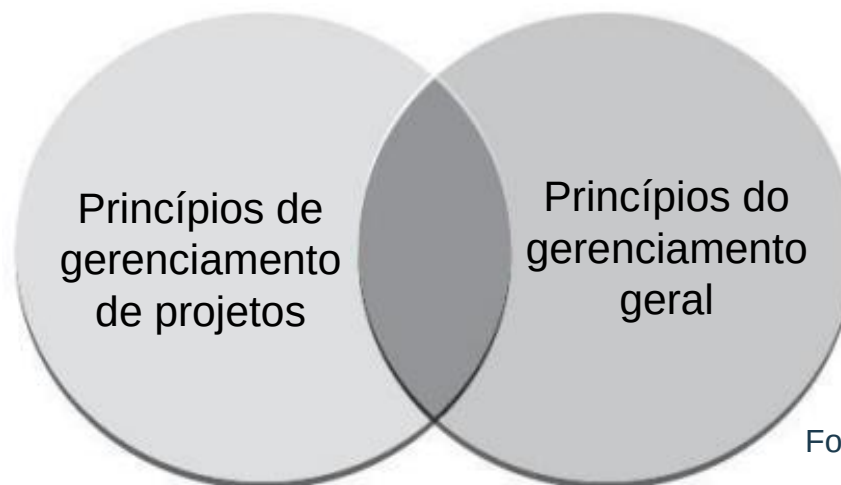
Fonte: Pressman (2021).

Sete características pessoais que demonstram competência:

1. Responsabilidade individual
2. Consciência aguçada
3. Extremamente honesto
4. Resiliência sob pressão
5. Lealdade
6. Atenção em detalhes
7. Pragmático

Análise de recursos para os processos do projeto

- O Guia PMBOK ® (2021) orienta que na gerência de projetos pode ter áreas de sobreposições que ele chama de “Princípios de gerenciamento de projetos” e “Princípios do gerenciamento geral”.
- O princípio de gerenciamento de projetos é praticado pela gerência administrativa, inclui processos para identificar, adquirir e gerenciar os recursos necessários para a conclusão bem-sucedida do projeto, bem como comprar ou adquirir produtos, serviços ou resultados externos à equipe do projeto e subcontratações. Esse perfil é do gerente de projeto.
- O princípio do gerenciamento geral ligado à engenharia de software é praticado pela gerência da tecnologia da informação (TI), que se refere ao gerenciamento do projeto do produto software. Esse perfil é do gerente da TI (ou do Sistema).



Fonte: Moreno (2020).

Recursos do projeto e do produto software I

- Um documento breve serve de instrumento de comunicação para avaliar a organização das pessoas envolvidas no projeto de software (stakeholders), definindo papéis e responsabilidades de acordo com a capacitação técnica e aptidões.
- Para cada pessoa envolvida no projeto, o gestor avalia e determina suas atividades, datas, custos e recursos necessários.
- Na projeção dos recursos para os processos do projeto, três perfis de analistas são essenciais: analista de negócios, analista de processo e analista de sistema.
- Analista de negócio (ou autor do processo) – tem a ideia do negócio e do que precisa da tecnologia da informação.
 - Analista de processo – é aquele que interpreta a ideia do negócio, seus riscos e regras, e que tem por objetivo determinar as atividades para processar o negócio.
 - Analista de sistema – determina os componentes do sistema e o workflow que levam à criação das funções do software.

Recursos do projeto e do produto software II

- Os objetivos da análise de recursos são:
- Garantir as metas do projeto: Avaliar se os recursos disponíveis atendem às necessidades para atingir os objetivos do projeto.
- Minimizar riscos: Identificar lacunas ou pontos críticos nos recursos que possam comprometer prazos, custos ou qualidade do produto.
- Aumentar a eficiência: Alocar recursos de forma otimizada para evitar desperdícios e sobrecarga da equipe.
- Com base nos requisitos, esta abordagem recai em duas categorias gerenciais: gerência do projeto e gerência do produto software.

Gerente de projeto	Gerente do produto software
Recursos humanos	Recursos de software
Recursos materiais	Recursos de hardware
Recursos tecnológicos	Recursos humanos (peopleware)
Recursos financeiros	Recursos de dados
	Recursos de rede de computadores

Interatividade

Para garantir a execução do projeto de software, analistas, gestores de TI e gerentes de projeto devem lidar com recursos distintos, sob sua responsabilidade. Abaixo estão alguns exemplos:

- I. Contratação de equipe de desenvolvedores para o projeto.
- II. Configuração dos servidores de aplicação e banco de dados.
- III. Estimativa de esforço e cronograma de entregas.
- IV. Definição das políticas de segurança da informação.
- V. Acompanhamento das entregas dos módulos do sistema.

Identifique corretamente o responsável de cada item, assinalando a alternativa correspondente:

- a) Gerente do projeto – I, II e IV; Gerente da TI – III e V.
- b) Gerente do projeto – I, II e V; Gerente da TI – III e IV.
- c) Gerente do projeto – I, III e V; Gerente da TI – II e IV.
- d) Gerente do projeto – II, III e IV; Gerente da TI – I e V.
- e) Gerente do projeto – III e IV; Gerente da TI – I, II e V.

Resposta

Para garantir a execução do projeto de software, analistas, gestores de TI e gerentes de projeto devem lidar com recursos distintos, sob sua responsabilidade. Abaixo estão alguns exemplos:

- I. Contratação de equipe de desenvolvedores para o projeto.
- II. Configuração dos servidores de aplicação e banco de dados.
- III. Estimativa de esforço e cronograma de entregas.
- IV. Definição das políticas de segurança da informação.
- V. Acompanhamento das entregas dos módulos do sistema.

Identifique corretamente o responsável de cada item, assinalando a alternativa correspondente:

- a) Gerente do projeto – I, II e IV; Gerente da TI – III e V.
- b) Gerente do projeto – I, II e V; Gerente da TI – III e IV.
- c) Gerente do projeto – I, III e V; Gerente da TI – II e IV.
- d) Gerente do projeto – II, III e IV; Gerente da TI – I e V.
- e) Gerente do projeto – III e IV; Gerente da TI – I, II e V.

Recursos do projeto sob a gerência de projetos

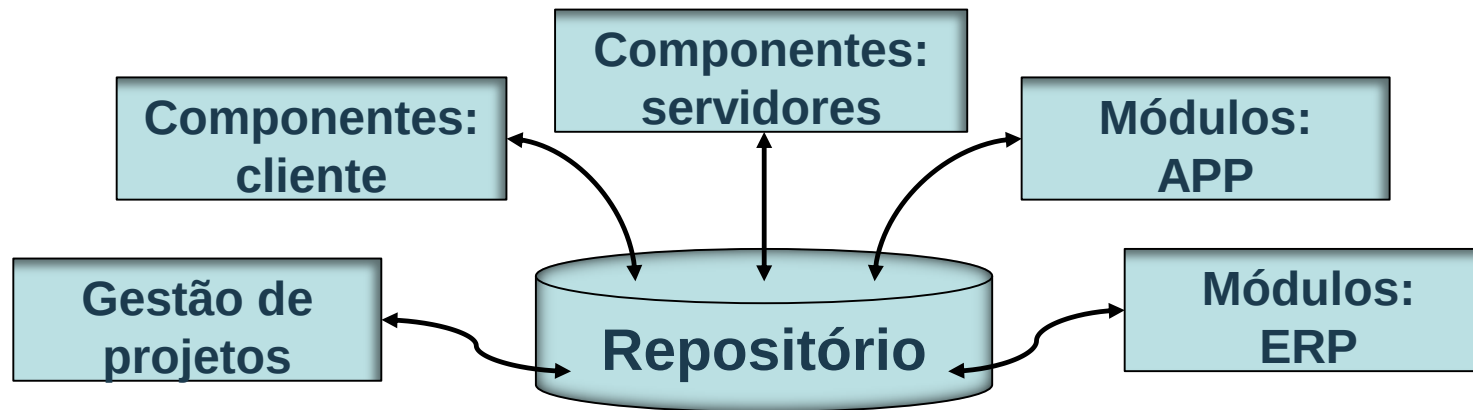
- Recursos humanos: Incluem todos os stakeholders alinhados ao negócio com práticas de gestão administrativa e representantes do cliente como o próprio gerente do projeto, coordenadores que auxiliam na comunicação, analistas do negócio e do processo, contratação de fornecedores, consultores e outros, promove o projeto, compras de suprimentos, treinamento, capacitações e workshops.
- Recursos materiais: Se referem à localização, transporte, mobília e instalações para o ambiente de uso e ambiente de desenvolvimento do software.
- Recursos tecnológicos: Envolvem os recursos consumidos pela gerência de TI, como ferramentas de software, licenças, ambiente de desenvolvimento e infraestrutura de TI.
 - Recursos financeiros: Fornecem o financiamento para cobrir os custos associados ao desenvolvimento do software, incluindo salários, ferramentas, infraestrutura e outros.

Recursos do projeto sob a gerência da TI

- Recursos de software: Referem-se às ferramentas, plataformas e soluções digitais fáceis para o desenvolvimento, gestão, implantação e manutenção de sistemas e aplicativos. Esses recursos desempenham um papel essencial nas etapas do ciclo de vida do desenvolvimento.
- Recursos de hardware: São recursos que garantem o desempenho necessário para executar software, oferece suporte para o desenvolvimento de sistemas escaláveis e seguros e que possibilitem conexões e operações em ambientes corporativos.
- Recursos humanos (peopleware): O termo peopleware diz respeito às pessoas envolvidas no uso, desenvolvimento e gestão da tecnologia.
 - Recursos de dados: É qualquer tipo de informação ou dados que podem ser usados para análises, tomada de decisões, ou para alimentar sistemas de software.
 - Recursos da rede de computadores: São os serviços e componentes associados que permitem a conectividade.

Métodos, ferramentas e técnicas (MF&T)

- No projeto de software basicamente são desenvolvidas três arquiteturas: 1. arquitetura da lógica de processamento; 2. arquitetura da informação; e 3. arquitetura da infraestrutura de TI.
- Estas arquiteturas compõem um repertório de alternativas de componentes para construir software, que ficam armazenados em um repositório (armazenamento de “peças”).

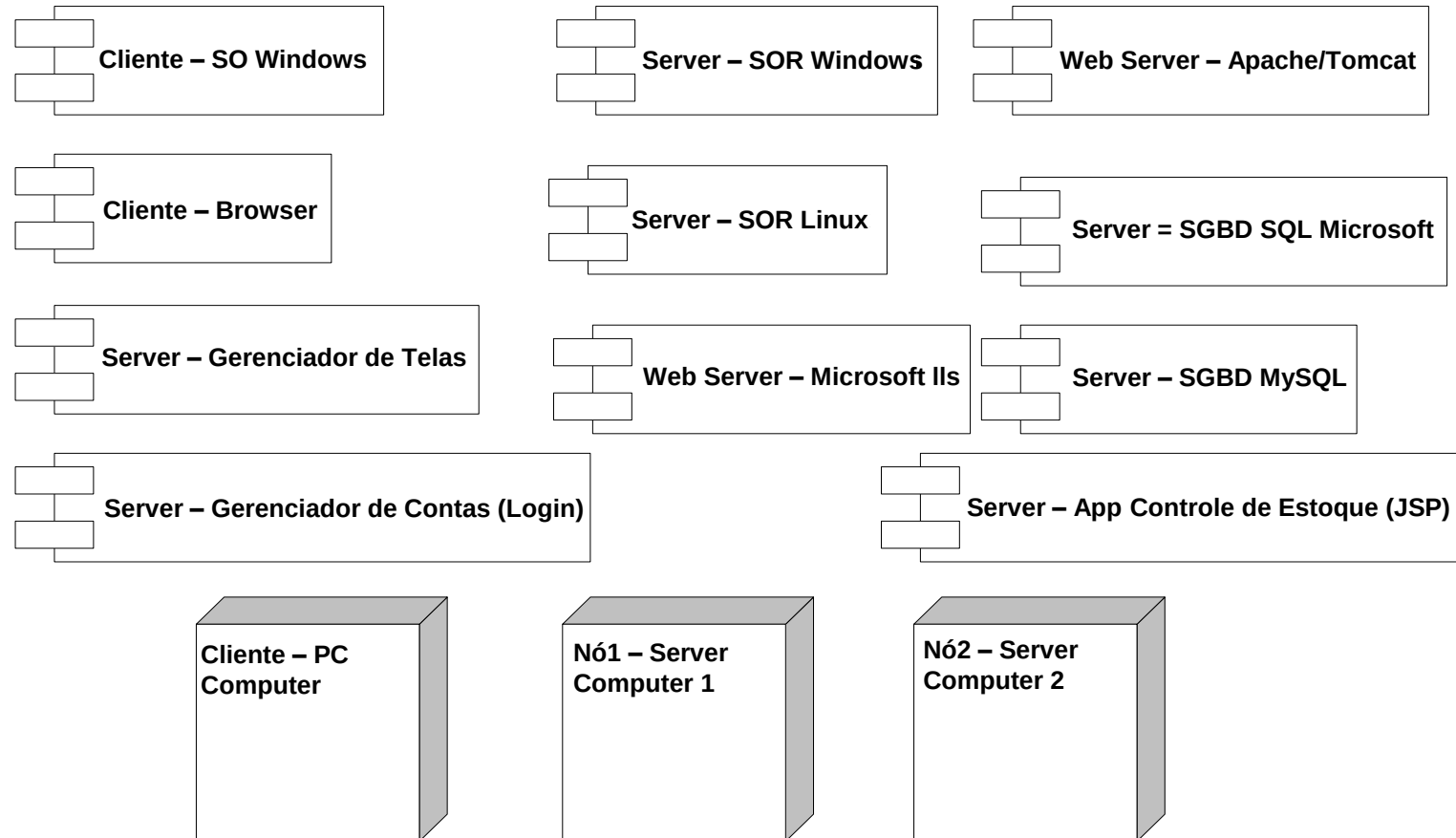


Fonte: Moreno (2023).

- A reusabilidade de componentes de software é um princípio da engenharia de software, usada para avaliar o quanto um programa ou parte dele pode ser usado em outras aplicações.
- O componente de software é uma unidade modular e independente de código externo.

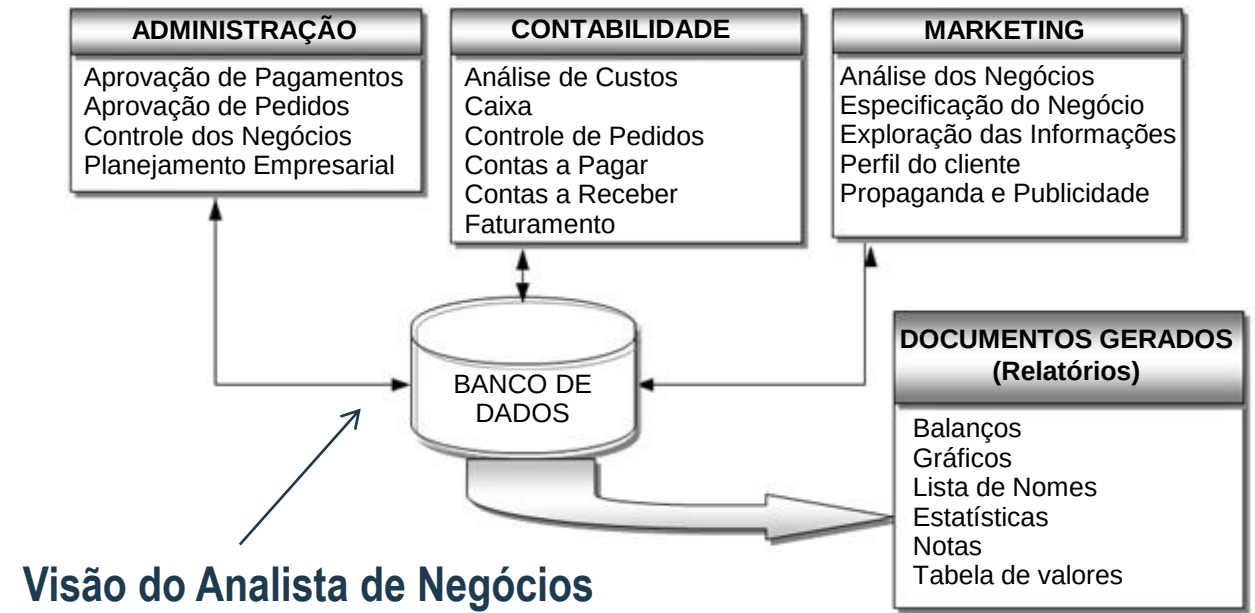
Métodos, ferramentas e técnicas (MF&T): Seleção de componentes

- A modelagem de construção com base no diagrama de componentes/implantação é a prática da convergência, em que o projetista escolhe os elementos do repertório que satisfaça os requisitos do sistema (RS) e os resume em uma particular configuração de componentes encapsulada para a criação do produto final.
- Ao lado, vemos alguns dos componentes disponíveis para o projeto de software.

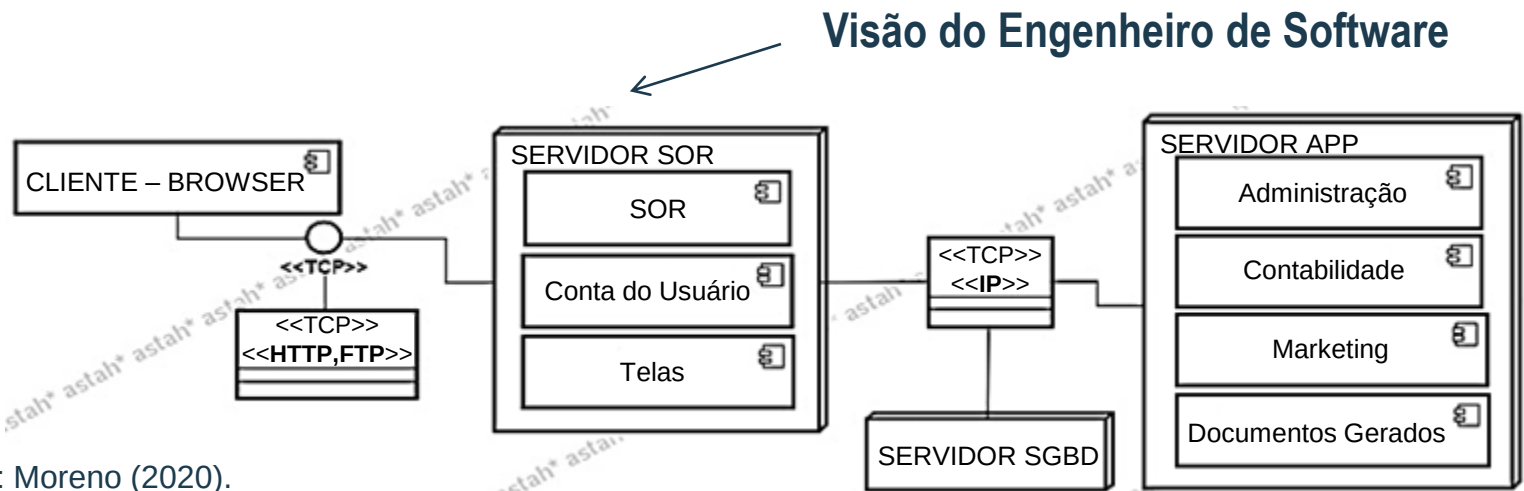


Métodos, ferramentas e técnicas (MF&T): Componentização

- No ERP as avaliações das características de capacidade e usabilidade do software são feitas por módulos.
- Na análise do negócio, o princípio básico é dividir esse sistema em módulos. Isto é chamado de modularidade.
- Na análise do sistema, um módulo surge a partir do momento em que cada componente que o constitui adquire sua independência funcional.



Visão do Analista de Negócios



Interatividade

Abaixo estão descritas algumas atividades no contexto do gerenciamento de projetos de TI:

- I. Escrever e desenvolver o código do software para manipulação de dados de acordo com as especificações técnicas.
- II. Fornecer feedbacks e aprovação em fases-chave do desenvolvimento.
- III. Planejar e definir objetivos, escopo, prazos e recursos do projeto.

Assinale a alternativa que corresponde aos perfis profissionais responsáveis pela atividade:

- a) I – Administrador do banco de dados; II – Cliente; III – Analista de sistemas.
- b) I – Administrador do banco de dados; II – Engenheiro de software; III – Gerente do projeto.
- c) I – Analista de sistemas; II – Engenheiro de software; III – Analista de sistemas.
- d) I – Programador; II – Gerente de projeto; III – Cliente.
- e) I – Programador; II – Cliente; III – Gerente do projeto.

Resposta

Abaixo estão descritas algumas atividades no contexto do gerenciamento de projetos de TI:

- I. Escrever e desenvolver o código do software para manipulação de dados de acordo com as especificações técnicas.
- II. Fornecer feedbacks e aprovação em fases-chave do desenvolvimento.
- III. Planejar e definir objetivos, escopo, prazos e recursos do projeto.

Assinale a alternativa que corresponde aos perfis profissionais responsáveis pela atividade:

- a) I – Administrador do banco de dados; II – Cliente; III – Analista de sistemas.
- b) I – Administrador do banco de dados; II – Engenheiro de software; III – Gerente do projeto.
- c) I – Analista de sistemas; II – Engenheiro de software; III – Analista de sistemas.
- d) I – Programador; II – Gerente de projeto; III – Cliente.
- e) I – Programador; II – Cliente; III – Gerente do projeto.

Referências

- ARKAN. Descubra como funciona o desenvolvimento de softwares e seus processos. *Arkan System*. Disponível em: https://arkansystem.com.br/desenvolvimento-de-softwares_e_seus_processos/. Acesso em: 26 jul. 2025.
- HAYES, Will; OVER, James W. *The Personal Software Process (PSP): An Empirical Study of the Impact of PSP on Individual Engineers*. Pennsylvania, USA: CMU/SEI, 1997. (76p.). Disponível em: https://insights.sei.cmu.edu/documents/1166/1997_005_001_16565.pdf?form=MG0AV3. Acesso em: 21 jan. 2025.
- PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH 2016. (968 p.)
 - PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de software: uma abordagem profissional*. 9. ed. Porto Alegre: AMGH, 2021. (1348 p.)
 - SOMMERVILLE, Ian. *Engenharia de software*. 9. ed. São Paulo: Pearson Prentice H., 2011.

ATÉ A PRÓXIMA!